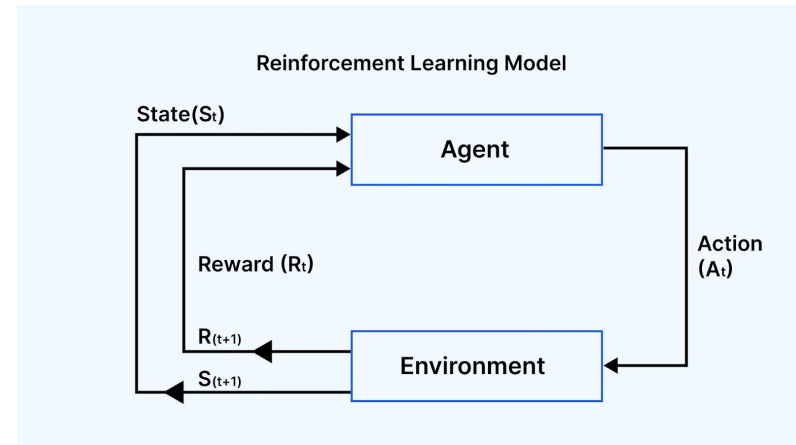
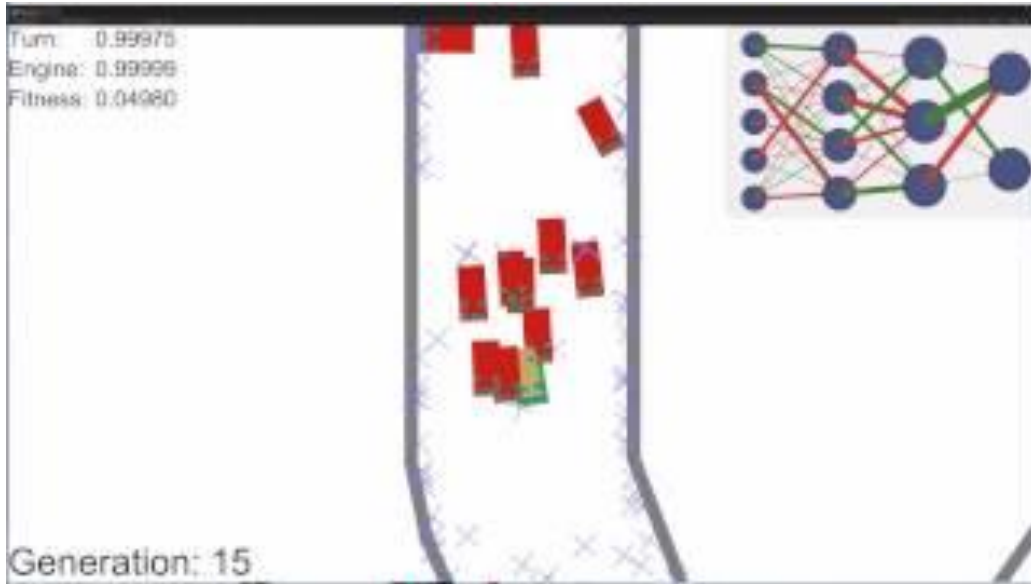


Session 4: Reinforcement Learning from Verifiable Rewards

Reinforcement Learning

- Agent performs a task in an environment and given a reward at each step.
- Agent's objective is to maximize it's cumulative reward.
- The strategy agent learns to achieve this is called a policy.
- The strategy which yields the maximum cumulative reward is called the optimal policy.



RL Setup

- At each step agent predicts the cumulative reward of each trajectory it can take, pursues the trajectory which it think will give max cumulative reward.
- Gets reward from the env at each step and updates its policy.

Agent: the cat

State: the position of the cat (x, y) in the grid

Action: at each position, the cat can move to one of the 4-directionally connected cells. If a move is invalid, the cell will not move and remain in the same position. Every time the cat makes a move, it results in a new state and a reward.

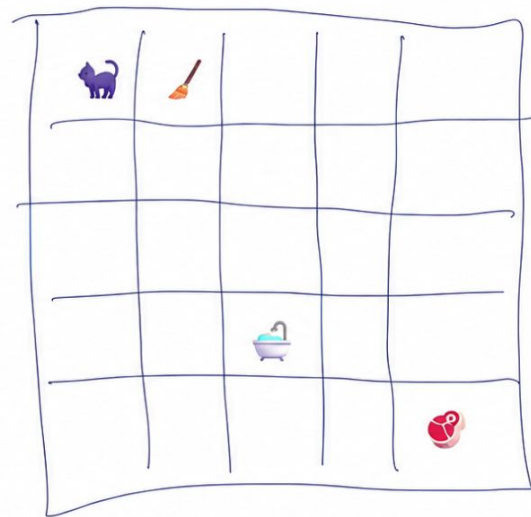
Reward model:

- A move to another empty cell results in a reward of 0.
- A move towards the broom, will result in a reward of -1.
- A move towards the bathtub will result in a reward of -10 and the cat fainting (episode over). The cat will be respawned at the initial position again.
- A move towards the meat will result in a reward of +100

Policy: a policy rules how the agent selects the action to perform given the state it is in;

$$a_t \sim \pi(\cdot | s_t)$$

The goal in RL is to select a policy that maximizes the expected return when the agent acts according to it.



RL Setup for LLM

Agent: the language model itself

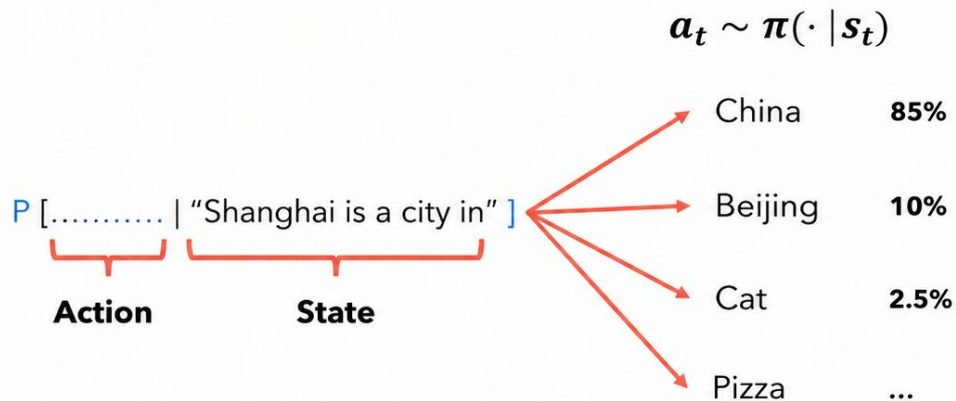
State: the prompt (input tokens)

Action: which token is selected as the next token

Reward model: the language model should be rewarded for generating “good responses” and should not receive any reward for generating “bad responses”.

Policy: In the case of language models, the policy is the language model itself! Because it models the probability of the action space given the current state of the agent: $a_t \sim \pi(\cdot | s_t)$

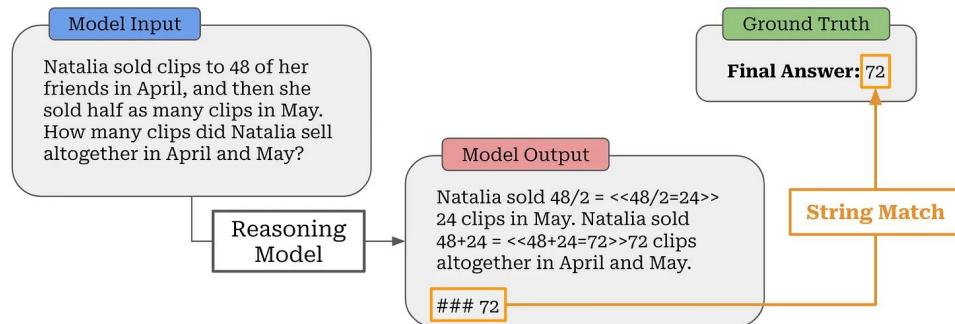
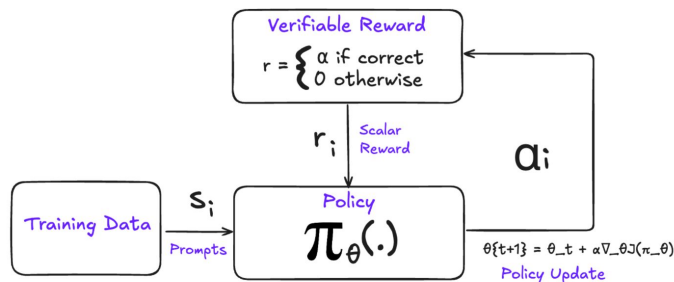
Let's look at how we can define the reward model for our language model



Reinforcement Learning from Verifiable Rewards (RLVR)

- RLHF: Collect preference data -> Train a reward model -> Give rewards to LLM.
- RLVR: Model generates 1 or more responses -> a deterministic verifier check for answer and assigns a reward -> Algo then updates the model to max reward.

Reinforcement Learning with Verifiable Rewards



Verifiable Domains

- Domains where correctness can be objectively verified.
- Verifiable domains: Math, Coding, Logic, SQL, MCQ.
- Non verifiable domains: Law, Financial, Poetry, Medical, Storytelling.
- Anything we can measure will be solved.

Asymmetry of verification can be improved with privileged information



Verifier's rule: The ease of training AI to solve a task is proportional to how verifiable the task is. All tasks that are possible to solve and easy to verify will be solved by AI.

More specifically, the ability to train AI to solve a task is proportional to whether the task has the following properties:

1. Objective truth: everyone agrees what good solutions are
2. Fast to verify: any given solution can be verified in a few seconds
3. Scalable to verify: many solutions can be verified simultaneously
4. Low noise: verification is as tightly correlated to the solution quality as possible
5. Continuous reward: it's easy to rank the goodness of many solutions for a single problem

Group Relative Policy Optimization (GRPO)

For each question q , GRPO samples a group of outputs $\{o_1, o_2, \dots, o_G\}$ from the old policy $\pi_{\theta_{old}}$ and then optimizes the policy model π_{θ} by maximizing the following objective:

$$\mathcal{J}_{GRPO}(\theta) = \mathbb{E}[q \sim P(Q), \{o_i\}_{i=1}^G \sim \pi_{\theta_{old}}(O|q)]$$
$$\frac{1}{G} \sum_{i=1}^G \left(\min \left(\frac{\pi_{\theta}(o_i|q)}{\pi_{\theta_{old}}(o_i|q)} A_i, \text{clip} \left(\frac{\pi_{\theta}(o_i|q)}{\pi_{\theta_{old}}(o_i|q)}, 1 - \varepsilon, 1 + \varepsilon \right) A_i \right) - \beta \mathbb{D}_{KL}(\pi_{\theta} || \pi_{ref}) \right), \quad (1)$$

$$\mathbb{D}_{KL}(\pi_{\theta} || \pi_{ref}) = \frac{\pi_{ref}(o_i|q)}{\pi_{\theta}(o_i|q)} - \log \frac{\pi_{ref}(o_i|q)}{\pi_{\theta}(o_i|q)} - 1, \quad (2)$$

where π_{ref} is a reference policy, ε and β are hyper-parameters, and A_i is the advantage, computed using a group of rewards $\{r_1, r_2, \dots, r_G\}$ corresponding to the outputs within each group:

$$A_i = \frac{r_i - \text{mean}(\{r_1, r_2, \dots, r_G\})}{\text{std}(\{r_1, r_2, \dots, r_G\})}. \quad (3)$$

GRPO - Advantage

Equation:

$$A_i = \frac{R_i - \mu}{\sigma + \epsilon}$$

where: $\mu = \frac{1}{G} \sum_{i=1}^G R_i$ and $\sigma = \text{std}(R_1, R_2, \dots, R_G)$

Explanation:

- R_i : The composite scalar reward score evaluated for completion rollout y_i using verifiable format and correctness functions.
- μ : The mean reward score calculated across all G completions generated for prompt x . Replaces the traditional Critic network.
- σ : The standard deviation of rewards within the sampled group G , normalizing the spread of scores.
- $A_i > 0$ (**Positive Advantage**): Completion y_i scored **above the group baseline** μ (performing better than peer rollouts).
- $A_i < 0$ (**Negative Advantage**): Completion y_i scored **below the group baseline** μ (performing worse than peer rollouts).
- $A_i = 0$ (**Zero Advantage**): Completion y_i scored **exactly at the group baseline** μ .

GRPO - Probability Ratio

Equation:

$$r_{i,t}(\theta) = \frac{\pi_{\theta}(y_{i,t} \mid x, y_{i,<t})}{\pi_{\theta_{\text{old}}}(y_{i,t} \mid x, y_{i,<t})}$$

Explanation:

- $y_{i,t}$: The specific token generated at step t in completion y_i (includes reasoning text inside thinking tags, structural XML formatting, and final answer tokens).
- $\pi_{\theta}(y_{i,t} \mid x, y_{i,<t})$: Probability assigned to token $y_{i,t}$ by the current active policy π_{θ} being updated during training.
- $\pi_{\theta_{\text{old}}}(y_{i,t} \mid x, y_{i,<t})$: Probability assigned to token $y_{i,t}$ by the sampling policy $\pi_{\theta_{\text{old}}}$ that generated the rollout batch.
- $r_{i,t}(\theta) > 1.0$: The active policy π_{θ} assigns a **higher probability** to token $y_{i,t}$ than the old policy (the token became more likely under current weights θ).
- $r_{i,t}(\theta) < 1.0$: The active policy π_{θ} assigns a **lower probability** to token $y_{i,t}$ than the old policy (the token became less likely under current weights θ).
- $r_{i,t}(\theta) = 1.0$: Active policy π_{θ} and old policy $\pi_{\theta_{\text{old}}}$ assign identical probability to token $y_{i,t}$.

GRPO - Probability Ratio Clipping

Equation:

$$\text{Clipped Ratio} = \text{clip}(r_{i,t}(\theta), 1 - \epsilon_{\text{low}}, 1 + \epsilon_{\text{high}})$$

Explanation:

- **Purpose:** Restricts the magnitude of token probability adjustments per optimizer step to guarantee conservative, stable policy updates.
- $1 + \epsilon_{\text{high}}$ (**Upper Bound**): Applied when advantage is positive ($A_i > 0$). Caps the maximum allowed probability ratio growth at $1 + \epsilon_{\text{high}}$. Prevents a single high-reward rollout from causing excessive gradient steps.
- $1 - \epsilon_{\text{low}}$ (**Lower Bound**): Applied when advantage is negative ($A_i < 0$). Floors the maximum allowed probability ratio drop at $1 - \epsilon_{\text{low}}$. Prevents policy collapse caused by over-penalizing bad rollouts.

GRPO - Pessimistic Token Objective

Equation:

$$\text{Objective}_{i,t}(\theta) = \min(r_{i,t}(\theta)A_i, \text{clip}(r_{i,t}(\theta), 1 - \epsilon_{\text{low}}, 1 + \epsilon_{\text{high}})A_i)$$

Explanation:

- **$\min(\cdot)$ Operator:** Takes the lower (pessimistic) bound between unclipped and clipped advantage objectives.
- **Positive Advantage Case ($A_i > 0$):** If ratio $r_{i,t} > 1 + \epsilon_{\text{high}}$, the $\min(\cdot)$ operator selects the clipped ceiling $(1 + \epsilon_{\text{high}})A_i$, ignoring further probability increases.
- **Negative Advantage Case ($A_i < 0$):** If ratio $r_{i,t} < 1 - \epsilon_{\text{low}}$, the $\min(\cdot)$ operator selects the clipped floor $(1 - \epsilon_{\text{low}})A_i$, ignoring further probability decreases.
- **Token-Level Scaling:** Applies scalar advantage A_i across every token position $t = 1 \dots |y_i|$ in completion y_i .

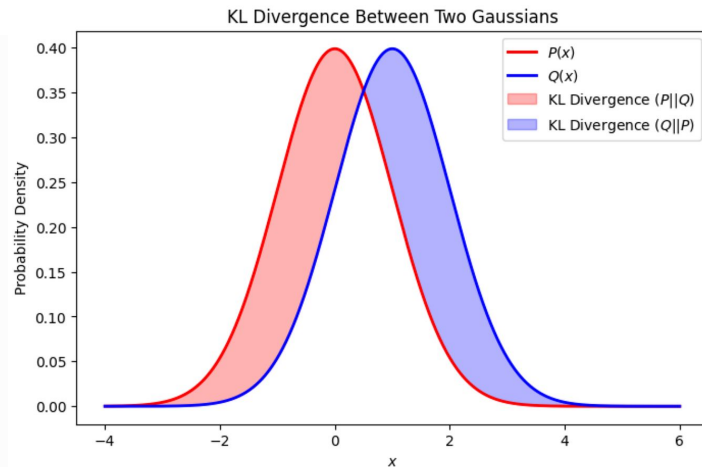
GRPO - KL Divergence Penalty

$$-\beta \mathbb{D}_{\text{KL}}(\pi_{\theta} \parallel \pi_{\text{ref}})$$

Where:

$$\mathbb{D}_{\text{KL}}(\pi_{\theta} \parallel \pi_{\text{ref}}) = \frac{\pi_{\text{ref}}(o_i|q)}{\pi_{\theta}(o_i|q)} - \log \frac{\pi_{\text{ref}}(o_i|q)}{\pi_{\theta}(o_i|q)} - 1$$

- π_{ref} : The frozen reference model (usually the original base model before RL fine-tuning).
- $\mathbb{D}_{\text{KL}}(\cdot)$: A specialized, unbiased estimator of Kullback–Leibler divergence (Schulman estimator). It measures how far the active policy π_{θ} strays from the original model π_{ref} .
- β : A hyperparameter coefficient that controls how strongly the model is penalized for drifting too far from π_{ref} (preventing reward hacking and degeneration).



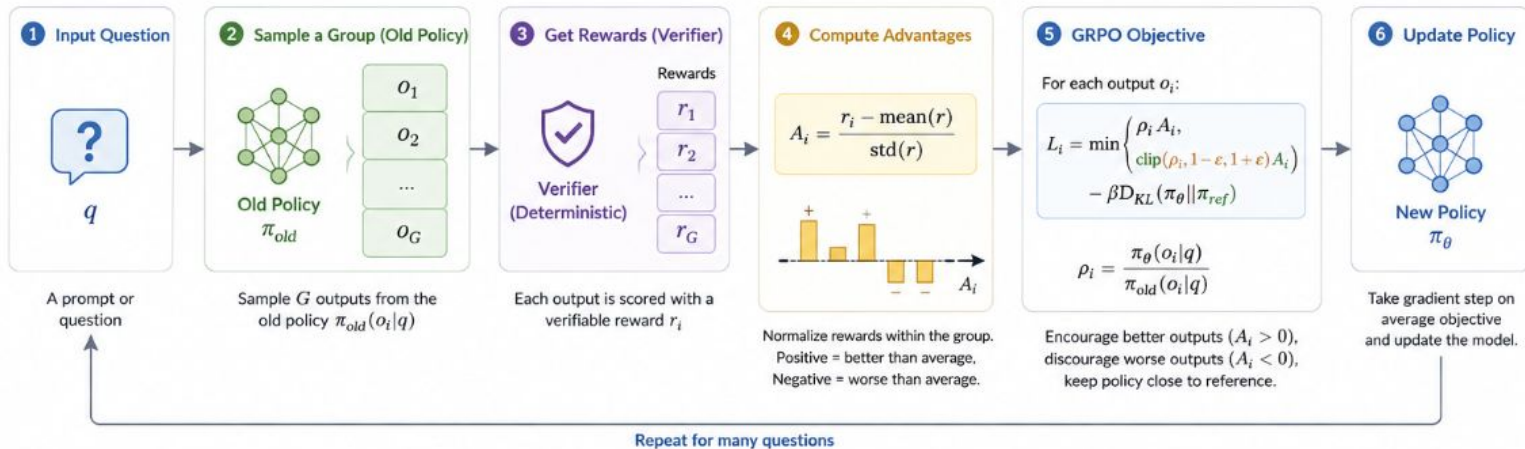
GRPO - Complete Loss Function

$$\mathcal{L}_{\text{GRPO}}(\theta) = -\frac{1}{G} \sum_{i=1}^G \frac{1}{|y_i|} \sum_{t=1}^{|y_i|} [\min(r_{i,t}(\theta)A_i, \text{clip}(r_{i,t}(\theta), 1 - \epsilon_{\text{low}}, 1 + \epsilon_{\text{high}})A_i)] + \beta D_{\text{KL}}(\pi_{\theta} \parallel \pi_{\text{ref}})$$

Explanation:

- **Outer Summation** ($\frac{1}{G} \sum_{i=1}^G$): Averages token objectives across all G completion rollouts sampled for prompt x .
- **Inner Summation** ($\frac{1}{|y_i|} \sum_{t=1}^{|y_i|}$): Averages token objectives across all $|y_i|$ tokens within completion rollout y_i .
- **Minus Sign** (-): Converts policy reward maximization into standard loss minimization for PyTorch autograd:
 - For $A_i > 0$, loss is **negative** → Minimizing loss requires **increasing ratio** $r_{i,t} > 1.0$ (raising token probability).
 - For $A_i < 0$, loss is **positive** → Minimizing loss requires **decreasing ratio** $r_{i,t} < 1.0$ (lowering token probability).
- β : Scaling hyperparameter governing the strength of the reference policy KL constraint.

GRPO Training



Algorithm 1 Iterative Group Relative Policy Optimization

Input initial policy model $\pi_{\theta_{init}}$; reward models r_ϕ ; task prompts $\mathcal{D}$; hyperparameters ϵ, β, μ

- 1: policy model $\pi_\theta \leftarrow \pi_{\theta_{init}}$
- 2: **for** iteration = 1, ..., I **do**
- 3: reference model $\pi_{ref} \leftarrow \pi_\theta$
- 4: **for** step = 1, ..., M **do**
- 5: Sample a batch $\mathcal{D}_b$ from $\mathcal{D}$
- 6: Update the old policy model $\pi_{\theta_{old}} \leftarrow \pi_\theta$
- 7: Sample G outputs $\{o_i\}_{i=1}^G \sim \pi_{\theta_{old}}(\cdot | q)$ for each question $q \in \mathcal{D}_b$
- 8: Compute rewards $\{r_i\}_{i=1}^G$ for each sampled output o_i by running r_ϕ
- 9: Compute $\hat{A}_{i,t}$ for the t -th token of o_i through group relative advantage estimation.
- 10: **for** GRPO iteration = 1, ..., μ **do**
- 11: Update the policy model π_θ by maximizing the GRPO objective (Equation 21)
- 12: Update r_ϕ through continuous training using a replay mechanism.

Output π_θ

GRPO Objective (Average over the group)

$$\mathcal{J}_{GRPO}(\theta) = \mathbb{E}_{q \sim P(Q)} \left[\frac{1}{G} \sum_{i=1}^G \left(\min \left(\rho_i A_i, \text{clip}(\rho_i, 1 - \epsilon, 1 + \epsilon) A_i \right) - \beta D_{KL}(\pi_\theta || \pi_{ref}) \right) \right]$$

$\rho_i = \frac{\pi_\theta(o_i q)}{\pi_{old}(o_i q)}$ Probability ratio (New policy vs Old policy)	A_i Advantage (How much better/worse than average)	$\text{clip}(\rho_i, 1 - \epsilon, 1 + \epsilon)$ Clipping Prevents too large policy updates	$\beta D_{KL}(\pi_\theta \pi_{ref})$ KL Penalty Keeps policy close to the reference policy
---	---	---	--

Advantage Computation

$$A_i = \frac{r_i - \text{mean}(r_1, r_2, \dots, r_G)}{\text{std}(r_1, r_2, \dots, r_G)}$$

Where r_i is the reward of output o_i .

- $A_i > 0$: better than average $\rightarrow$ increase probability
- $A_i < 0$: worse than average $\rightarrow$ decrease probability

Emergent Behavior

- DeepSeek R1-Zero: DeepSeek V3 post-trained on GRPO.
- It started generating long reasoning traces and breaking down problems into smaller sub-problems spontaneously.
- The model began testing its assumptions, double-checking constraints and trying alternate paths for a failed calculation.
- The model naturally learned to allocate more processing tokens (longer chains of thought) as it tackles harder problems, because extended planning directly correlates with higher reward success.
- The model figured out how to use text tokens as "working memory" (a scratchpad space).

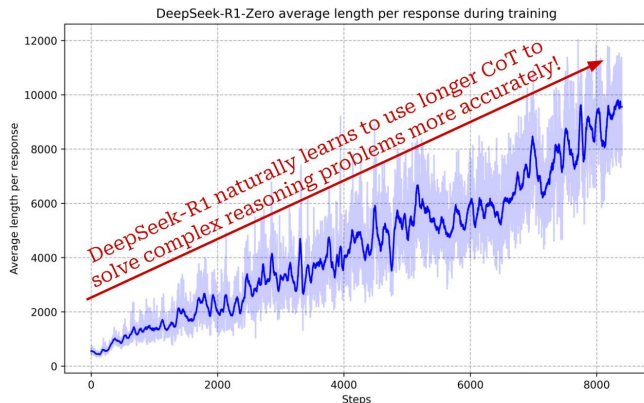


Figure 3 | The average response length of DeepSeek-R1-Zero on the training set during the RL process. DeepSeek-R1-Zero naturally learns to solve reasoning tasks with more thinking time.

Chain-of-Thought (CoT)

Decode this message: "Tango Romeo Echo November Delta (space) Mike India Charlie Romeo Oscar"

Exposed CoT

<think> Okay, let's see. I need to decode this message: "Tango Romeo Echo November Delta (space) Mike India Charlie Romeo Oscar". The user mentioned it's a message that needs decoding, so probably using some standard code. Since the words are all capitalized and separated by (space), I think each word corresponds to a letter in the NATO phonetic alphabet. Let me confirm that. First, I'll list out each word and its corresponding letter. Tango is T, Romeo is R, Echo is E, November is N, Delta is D. So the first part is TREND. Then after the space, Mike is M, India is I, Charlie is C, Romeo is R, Oscar is O. So the second part is MICRO. Putting them together, it's TREND MICRO. Wait, Trend Micro? That's a cybersecurity company. Maybe the message is just "Trend Micro". Let me check if there's any other way to interpret it. But the phonetic alphabet is straightforward. Each word represents a letter. So yes, combining them gives TREND MICRO. I don't see any hidden cipher here, like shifting letters or substitution. It's a direct phonetic alphabet translation. So the decoded message should be "Trend Micro".

The decoded message is "TREND MICRO".
Each word corresponds to a letter in the NATO phonetic alphabet:

- Tango → T
- Romeo → R
- Echo → E
- November → N
- Delta → D
- (space)
- Mike → M
- India → I
- Charlie → C
- Romeo → R
- Oscar → O

Combined, it spells "TREND MICRO", which is a well-known cybersecurity company.

©2025 TREND MICRO

Q1: Basic Algebra

A store sells notebooks for \$3 each and pens for \$2 each. Sarah bought some notebooks and twice as many pens, spending \$44 total. How many notebooks did she buy?

Thought for 33 seconds

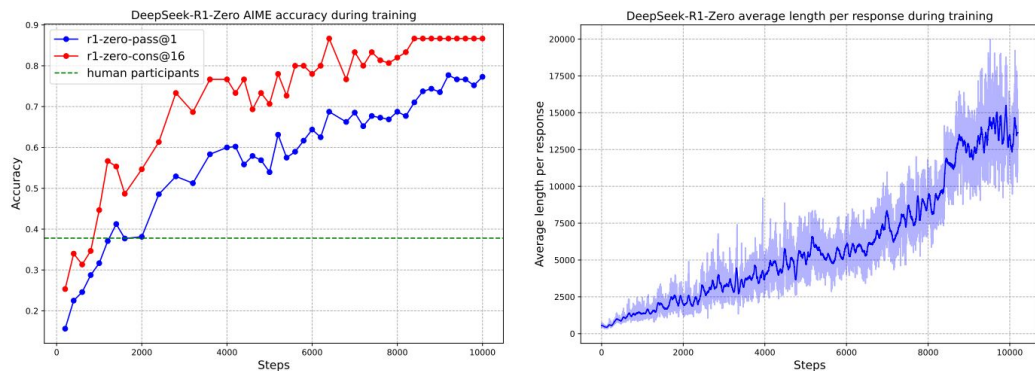
- We are asked: "A store sells notebooks for \$3 each and pens for \$2 each. Sarah bought some notebooks and twice as many pens, spending \$44 total. How many notebooks did she buy?"

Let n be the number of notebooks. Pens = $2n$. Total cost: $3n + 2(2n) = 3n + 4n = 7n = 44 \Rightarrow n = 44/7$ which is not an integer. $44/7 \approx 6.2857$. But number of notebooks must be integer. Something's wrong? Wait, "twice as many pens" means pens = $2 * \text{notebooks}$. $3n + 2(2n) = 7n = 44$. Not integer. Did I misread? "Sarah bought some notebooks and twice as many pens" - yes, pens = $2 * \text{notebooks}$. Total spent \$44. That gives $7n = 44$. The problem might have a typo? Or maybe "twice as many pens" means number of pens is twice the number of notebooks, but maybe she bought some notebooks and some pens, and the number of pens is twice the number of notebooks? Then $3n + 2*2n = 7n = 44 \Rightarrow n = 44/7$ not integer. Could it be that she bought some pens and twice as many notebooks? That would be: notebooks = $2p$, pens = $p \Rightarrow 3(2p) + 2p = 6p + 2p = 8p = 44 \Rightarrow p = 5.5$, not integer. Or maybe she bought "some pens and twice as many notebooks"? Still 5.5. Maybe total is different? Perhaps it's a trick and we need to accept fractional? No, usually these problems yield integer answers. Let me double-check problem statement: "A store sells notebooks for \$3 each and pens for \$2 each. Sarah bought some notebooks and twice as many pens, spending \$44 total. How many notebooks did she buy?" Could "twice as many pens" mean number of pens = $2 * \text{notebooks}$? $44/7$ is 6 remainder 2. Maybe she bought 6 notebooks and 12 pens = $18 + 24 = 42$, not 44. If she bought 7 notebooks and 14 pens = $21 + 28 = 49$. Nothing works. Maybe I misread the prices? Notebooks \$3, pens \$2. "twice as many pens" $\rightarrow 2n$ pens. Total = $3n + 4n = 7n$. $44/7$ is not integer. Could it be the total spent is \$42? Then $n=6$. Could the problem have a typo? Given that it's a basic algebra problem, maybe "twice as many notebooks as pens"?

Try out yourself with a reasoning ques: <https://chat.deepseek.com/>

DeepSeek R1-Zero

- DeepSeek V3 post-trained on GRPO.
- Emergent behavior but with problems: Poor readability, Inconsistent formatting, Weak general conversational abilities, difficult user pref alignment.
- Model learnt to reason but not how to communicate.



DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning

DeepSeek-AI
research@deepseek.com

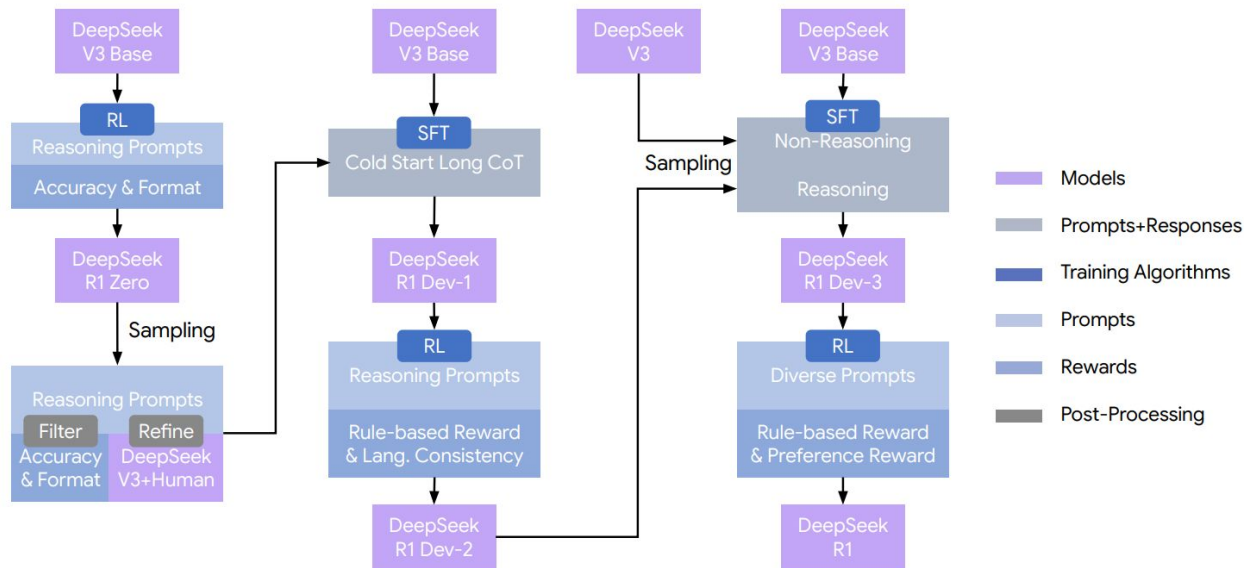
Abstract

General reasoning represents a long-standing and formidable challenge in artificial intelligence. Recent breakthroughs, exemplified by large language models (LLMs) (Brown et al., 2020; OpenAI, 2023) and chain-of-thought prompting (Wei et al., 2022b), have achieved considerable success on foundational reasoning tasks. However, this success is heavily contingent upon extensive human-annotated demonstrations, and models' capabilities are still insufficient for more complex problems. Here we show that the reasoning abilities of LLMs can be incentivized through pure reinforcement learning (RL), obviating the need for human-labeled reasoning trajectories. The proposed RL framework facilitates the emergent development of advanced reasoning patterns, such as self-reflection, verification, and dynamic strategy adaptation. Consequently, the trained model achieves superior performance on verifiable tasks such as mathematics, coding competitions, and STEM fields, surpassing its counterparts trained via conventional supervised learning on human demonstrations. Moreover, the emergent reasoning patterns exhibited by these large-scale models can be systematically harnessed to guide and enhance the reasoning capabilities of smaller models.

12948v2 [cs.CL] 4 Jan 2026

Figure 1 | (a) AIME accuracy of DeepSeek-R1-Zero during training. AIME takes a mathematical problem as input and a number as output, illustrated in Table 32. Pass@1 and Cons@16 are described in Supplementary D.1. The baseline is the average score achieved by human participants in the AIME competition. (b) The average response length of DeepSeek-R1-Zero on the training set during the RL process. DeepSeek-R1-Zero naturally learns to solve reasoning tasks with more thinking time. Note that a training step refers to a single policy update operation.

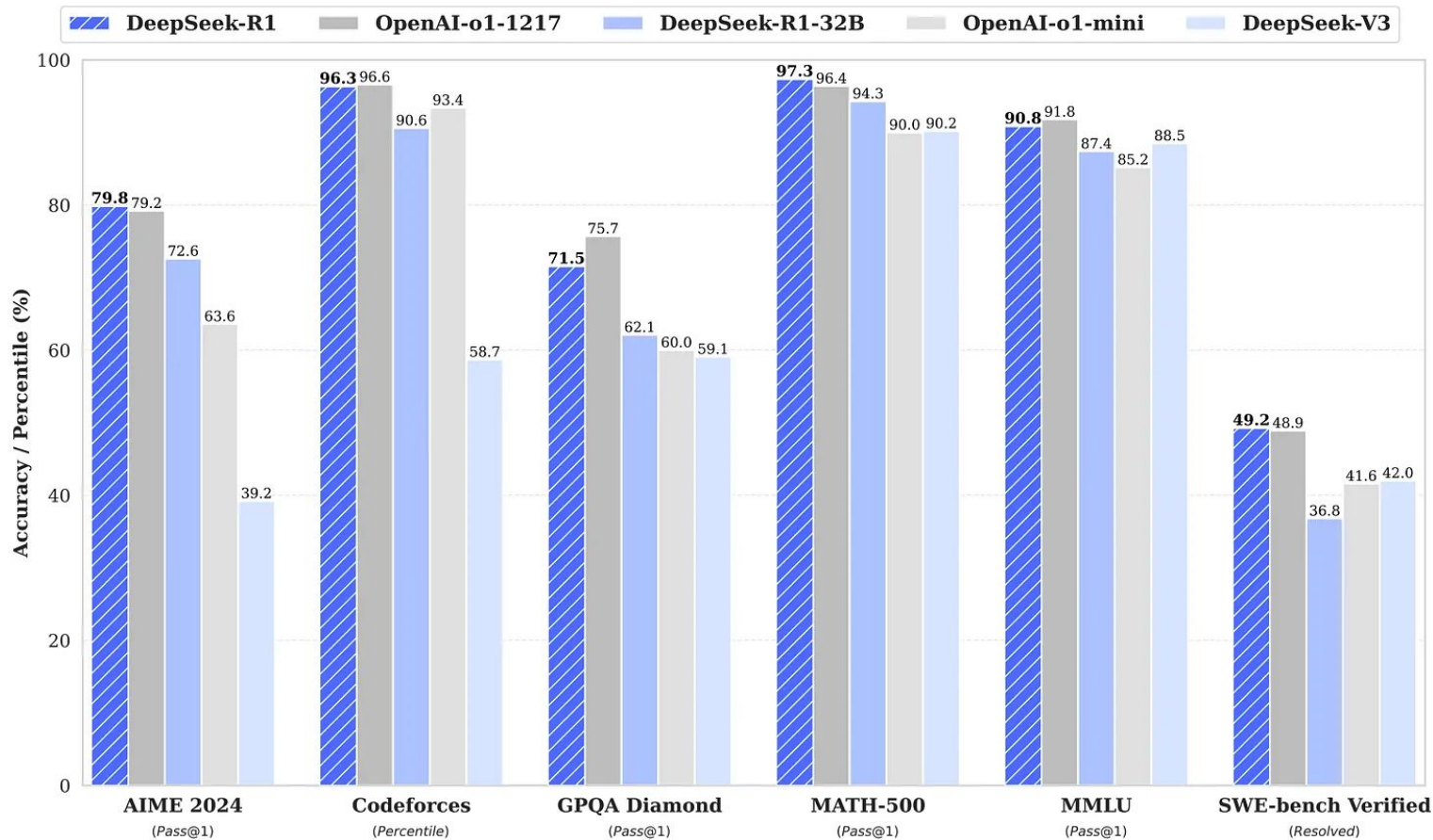
DeepSeek R1



	Benchmark (Metric)	R1-Zero	R1-Dev1	R1-Dev2	R1-Dev3	R1
English	MMLU (EM)	88.8	89.1	91.2	91.0	90.8
	MMLU-Redux (EM)	85.6	90.0	93.0	93.1	92.9
	MMLU-Pro (EM)	68.9	74.1	83.8	83.1	84.0
	DROP (3-shot F1)	89.1	89.8	91.1	88.7	92.2
	IF-Eval (Prompt Strict)	46.6	71.7	72.0	78.1	83.3
	GPQA Diamond (Pass@1)	75.8	66.1	70.7	71.2	71.5
	SimpleQA (Correct)	30.3	17.8	28.2	24.9	30.1
	FRAMES (Acc.)	82.3	78.5	81.8	81.9	82.5
	AlpacaEval2.0 (LC-winrate)	24.7	50.1	55.8	62.1	87.6
	ArenaHard (GPT-4-1106)	53.6	77.0	73.2	75.6	92.3
Code	LiveCodeBench (Pass@1-COT)	50.0	57.5	63.5	64.6	65.9
	Codeforces (Percentile)	80.4	84.5	90.5	92.1	96.3
	Codeforces (Rating)	1444	1534	1687	1746	2029
	SWE Verified (Resolved)	43.2	39.6	44.6	45.6	49.2
	Aider-Polyglot (Acc.)	12.2	6.7	25.6	44.8	53.3
Math	AIME 2024 (Pass@1)	77.9	59.0	74.0	78.1	79.8
	MATH-500 (Pass@1)	95.9	94.2	95.9	95.4	97.3
	CNMO 2024 (Pass@1)	88.1	58.0	73.9	77.3	78.8
Chinese	CLUEWSC (EM)	93.1	92.8	92.6	91.6	92.8
	C-Eval (EM)	92.8	85.7	91.9	86.4	91.8
	C-SimpleQA (Correct)	66.4	58.8	64.2	66.9	63.7

Figure 2 | The multi-stage pipeline of DeepSeek-R1. A detailed background on DeepSeek-V3 Base and DeepSeek-V3 is provided in Supplementary A.1. The models DeepSeek-R1 Dev1, Dev2, and Dev3 represent intermediate checkpoints within this pipeline.

DeepSeek R1 Benchmarks



Test-Time Scaling

- Previous paradigm: Train-Time scaling (larger models, more data, more compute).
- Human brain has 200T+ neuron connections. “Scaling is the way”.
- New/Current paradigm: Test-Time scaling (more computation is used during inference).
- Model doesn’t learn any new information rather builds reasoning.

